Experiment: Creating All Three Basic Gates Using NAND Gate

1. Problem 01

1.1 Problem Statement

The objective of this experiment is to design and implement the three basic logic gates, **AND, OR, and NOT**, using only **NAND gates**. Since the NAND gate is a universal logic gate, it can be used to construct other fundamental logic gates by properly connecting multiple NAND gates. The designed circuits are simulated and their outputs are verified using the corresponding truth tables.

1.2 Theory and Design Methodology

A **NAND gate** is a universal logic gate because all basic logic gates can be constructed using only NAND gates. In this experiment, the **NOT gate** is created by connecting both inputs of a NAND gate together. The **AND gate** is constructed by first performing a NAND operation on the two inputs and then passing the output through another NAND gate configured as a NOT gate. Similarly, the **OR gate** is implemented using De Morgan's theorem by first inverting both inputs using two NAND gates and then applying them to a third NAND gate. The output of each designed circuit is verified by comparing it with the standard truth table of the corresponding logic gate.

The Boolean expressions are:

NOT Gate:

$$Y = A \ NAND \ A = \overline{A}$$

AND Gate:

$$Y = (A \ NAND \ B) \ NAND \ (A \ NAND \ B)$$

$$Y = A \cdot B$$

OR Gate:

$$Y = (A \ NAND \ A) \ NAND \ (B \ NAND \ B)$$

$$Y = A + B$$

1.3 Circuit Diagram

The circuit diagrams for the **NOT, AND, and OR gates** are designed using only NAND gates. For the NOT gate, the two inputs of a single NAND gate are connected together. For the AND gate, two input signals are first connected to a NAND gate, and its output is connected to both inputs of a second NAND gate. For the OR gate, each input is first connected to a separate NAND gate

with its inputs tied together, and the two resulting outputs are then connected to a third NAND gate. These configurations demonstrate the universal property of the NAND gate and produce the same logical outputs as the conventional NOT, AND, and OR gates.

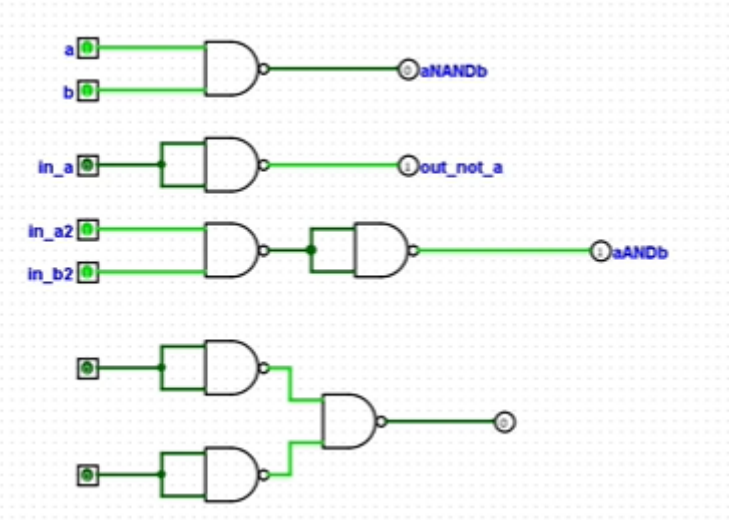

Figure 1: NOT Gate Using NAND

NOT,AND,OR Gate Using NAND

2. Problem 02

2.1 Problem Statement

The objective of this experiment is to design and implement the three basic logic gates, **AND, OR, and NOT**, using only **NOR gates**. Since the NOR gate is a universal logic gate, it can be used to construct other fundamental logic gates by properly connecting multiple NOR gates. The designed circuits are simulated and their outputs are verified using the corresponding truth tables.

2.2 Theory and Design Methodology

A **NOR gate** is a universal logic gate because all basic logic gates can be constructed using only NOR gates. In this experiment, the **NOT gate** is created by connecting both inputs of a NOR gate together. The **OR gate** is constructed by first performing a NOR operation on the two inputs and then passing the output through another NOR gate configured as a NOT gate. Similarly, the **AND gate** is implemented using De Morgan's theorem by first inverting both inputs using two NOR gates and then applying them to a third NOR gate. The output of each designed circuit is verified by comparing it with the standard truth table of the corresponding logic gate.

The Boolean expressions are:

NOT Gate:

$$Y = A \; NOR \; A = \overline{A}$$

OR Gate:

$$Y = (A\ NOR\ B)\ NOR\ (A\ NOR\ B)$$

$$Y = A + B$$

AND Gate:

$$Y = (A\ NOR\ A)\ NOR\ (B\ NOR\ B)$$

$$Y = A \cdot B$$

2.3 Circuit Diagram

The circuit diagrams for the **NOT, OR, and AND gates** are designed using only NOR gates. For the NOT gate, the two inputs of a single NOR gate are connected together. For the OR gate, two input signals are first connected to a NOR gate, and its output is connected to both inputs of a second NOR gate. For the AND gate, each input is first connected to a separate NOR gate with its inputs tied together, and the two resulting outputs are then connected to a third NOR gate. These configurations demonstrate the universal property of the NOR gate and produce the same logical outputs as the conventional NOT, OR, and AND gates.

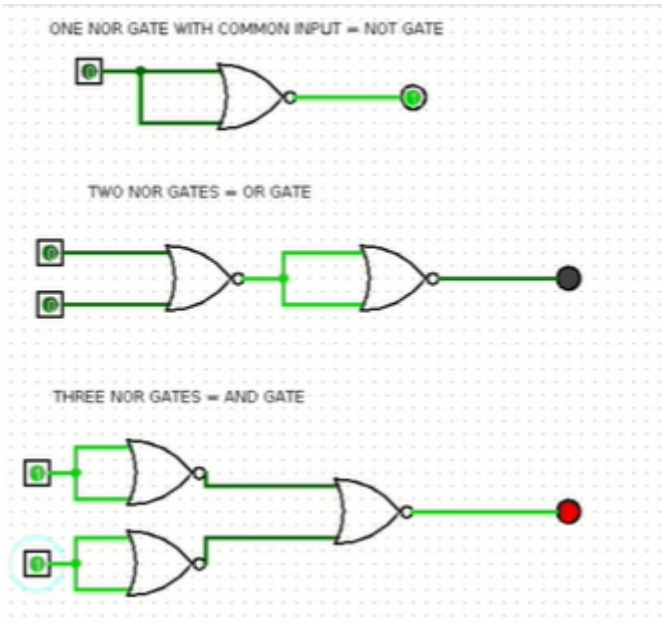

Figure 2: NOT Gate Using NOR

NOT,OR,AND Gate Using NOR

Experiment: Designing a Half Adder

3. Problem 03

3.1 Problem Statement

The objective of this experiment is to design and implement a **Half Adder** circuit using basic logic gates. A Half Adder is a combinational logic circuit that performs the addition of two single-bit binary inputs. The circuit produces two outputs: **Sum** and **Carry**. The designed circuit is simulated, and its outputs are verified using the corresponding truth table.

3.2 Theory and Design Methodology

A **Half Adder** is a combinational logic circuit used to add two single-bit binary numbers. It has two inputs, **A** and **B**, and two outputs, **Sum (S)** and **Carry (C)**. The **Sum** output is obtained using an **XOR gate**, while the **Carry** output is obtained using an **AND gate**. The circuit performs binary addition according to the rules of binary arithmetic. The output of the designed circuit is verified by comparing the simulation results with the standard Half Adder truth table.

The Boolean expressions are:

Sum:

$$S = A \oplus B$$

Carry:

$$C = A \cdot B$$

The truth table is:

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

3.3 Circuit Diagram

The Half Adder circuit is designed using one **XOR gate** and one **AND gate**. The two input signals, **A** and **B**, are connected to both gates simultaneously. The XOR gate produces the **Sum (S)** output, while the AND gate produces the **Carry (C)** output. This circuit demonstrates the basic principle of binary addition and produces the correct Sum and Carry outputs for all possible combinations of the two input bits.

Half Adder Circuit

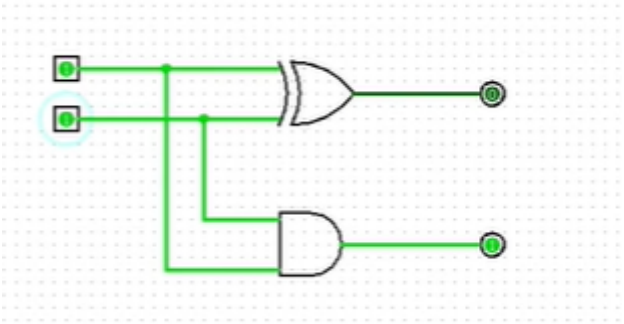

Figure 3: Half Adder Circuit

Experiment: Designing a Full Adder

4. Problem 01

4.1 Problem Statement

The objective of this experiment is to design and implement a **Full Adder** circuit using basic logic gates. A Full Adder is a combinational logic circuit that performs the addition of three single-bit binary inputs: **A**, **B**, and **Carry-in (Cin)**. The circuit produces two outputs: **Sum (S)** and **Carry-out (Cout)**. The designed circuit is simulated, and its outputs are verified using the corresponding truth table.

4.2 Theory and Design Methodology

A **Full Adder** is a combinational logic circuit used to add three single-bit binary inputs. It has three inputs: **A**, **B**, and **Carry-in (Cin)**, and two outputs: **Sum (S)** and **Carry-out (Cout)**. The Sum output is obtained using XOR operations, while the Carry-out output is generated using AND and OR operations. A Full Adder can also be implemented using **two Half Adders and one OR gate**. The output of the designed circuit is verified by comparing the simulation results with the standard Full Adder truth table.

The Boolean expressions are:

Sum:

$$S = A \oplus B \oplus C_{in}$$

Carry-out:

$$C_{out} = AB + C_{in}(A \oplus B)$$

The truth table is:

A	B	Cin	Sum (S)	Carry-out (Cout)
0	0	0	0	0
0	0	1	1	0

A	B	Cin	Sum (S)	Carry-out (Cout)
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4.3 Circuit Diagram

The Full Adder circuit is designed using **two XOR gates, two AND gates, and one OR gate**. The inputs **A** and **B** are first applied to an XOR gate to produce an intermediate Sum, which is then XORed with the **Carry-in (Cin)** to produce the final **Sum (S)** output. The carry generated from the first XOR operation is obtained using an AND gate, while another AND gate generates the carry from the intermediate Sum and **Cin**. The outputs of these two AND gates are then connected to an OR gate to produce the final **Carry-out (Cout)**. The circuit can also be implemented using two Half Adders and one OR gate.

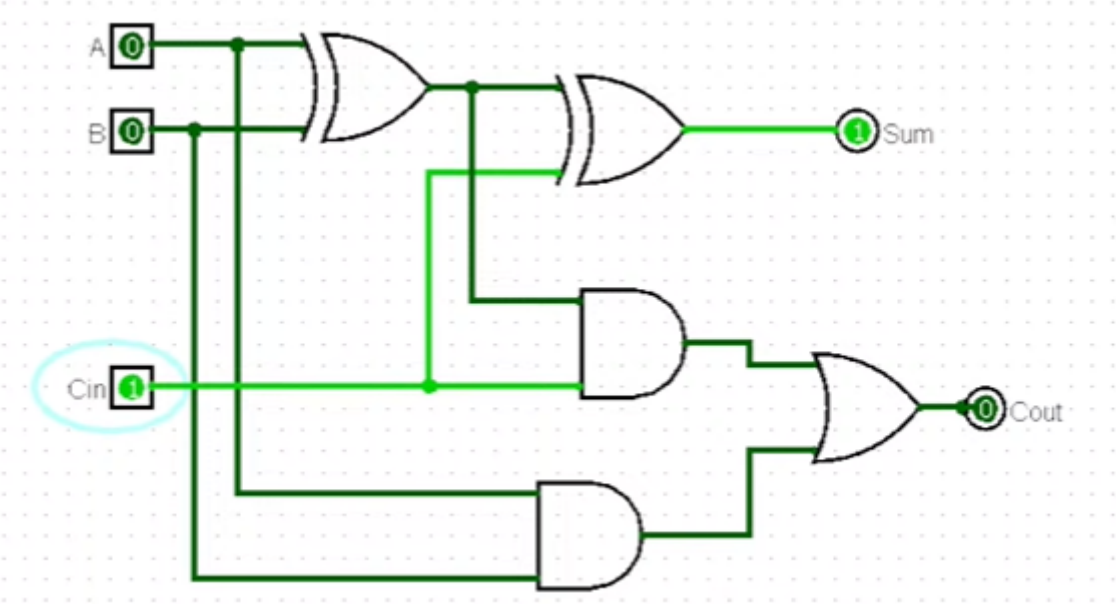

Figure 4: Full Adder Circuit

Full Adder Circuit